# base.py

`src/moe_pruning/tasks/base.py · 88 lines · module moe_pruning.tasks.base`

## What it does

`TaskObjective`: the interface for how a model family's data becomes model inputs and how the loss is computed. Plugins choose a task; the pipeline only calls this interface.

## How it works

- Defines calibration tokenization, batching, evaluation example construction, per-token loss, predictions and qualitative generation.
- `clean_decode()` decodes tokens while keeping special tokens such as T5 sentinels but removing pad/EOS noise.

## Defined in this file

Extracted from the source code (classes, public functions and methods).

| Name | Description |
|---|---|
| `class TaskObjective` | Interface: how a model family's data becomes inputs and how loss is computed. |
| `.tokenize_calibration(records, tokenizer, max_source_length, max_target_length)` | `human` is the input/prompt, `response` the teacher-forced target. |
| `.collate(items, pad_token_id)` | Right-padded batch with per-stack real-token masks. |
| `.iter_batches(items, batch_size, pad_token_id)` | Split items into collated batches. |
| `.eval_settings(cfg)` | Task-specific settings that determine the evaluation examples. |
| `.build_eval_example(example_id, text, tokenizer, cfg)` | One evaluation example (must expose `example_id`, `input_ids`, `labels`). |
| `.token_nll(logits, labels)` | Per-example (sum of token NLL, number of scored tokens). |
| `.predictions(logits, labels)` | Per-example argmax token at every scored position (same order as `token_nll`). |
| `.qualitative(model, tokenizer, examples, device, cfg)` | `[{"example_id", "inputs": {label: text}, "prediction"}]` from greedy generation. |
| `clean_decode(tokenizer, ids)` | Decode keeping special tokens such as T5 sentinels, but without pad/EOS noise. |

## Connections

Uses: `moe_pruning.data.calibration`, `moe_pruning.data.collators`

Used by: `src/moe_pruning/evaluation/evaluator.py`, `src/moe_pruning/models/__init__.py`, `src/moe_pruning/tasks/causal_lm.py`, `src/moe_pruning/tasks/seq2seq_denoising.py`